SAAP — Independent Architecture & Product Review

Reconstructed from the actual SRS (`Success_Academic_Analytics_Platform_Complete_IEEE_SRS.docx`) and the actual Semester-3 implementation ZIP. Not a courtesy review.

EXECUTIVE SUMMARY

The honest version, up front:

SAAP's documentation discipline is genuinely above average for a student project — the implementation is honest about being fake — but the *product concept* has not yet been forced to answer "so what happens differently because this exists?", and a handful of screens quietly mix real computed numbers with invented ones in a way that will read as dishonest, not illustrative, if it isn't fixed before Semester 4.

Three things are true at once:

1. **The SRS and prototype-review documentation are unusually disciplined.** The team already caught and corrected several of the exact failure modes this review would otherwise flag first — mislabeling a design assumption (75% attendance) as a statutory rule, overclaiming SHAP, and presenting simulated telemetry as if it were real. Section 14 of `prototype-review.md` is a self-conducted audit that most Semester-3 teams never do. That is a real, structural strength — it means the team has the *instinct* for rigor, not just the vocabulary of it.
2. **The current concept is "ERP + dashboards + AI branding," and the AI/ML is not yet earning its place.** Nothing in the current design closes the loop from an insight to an action to an outcome. SAAP detects and displays; it does not yet help anyone *do* anything about what it shows. That is the single biggest product gap, and it is a design problem, not an engineering one — it can't be fixed by adding a backend.
3. **The implementation contains silent fabrication sitting next to real computation, in the same screens.** `SCR002_StudentDashboard.jsx` computes real SGPA/attendance from the synthetic dataset, then two lines later hardcodes a "Cohort Rank: Top 15%" and a "+0.25 trend" that have nothing to do with any student's actual data. `SCR008_DepartmentAnalytics` computes a real average mark, then hardcodes a pass percentage by string-matching the subject code. This is more dangerous than the clearly-labeled simulations (ML metadata, telemetry) because it is *not* labeled, and a reviewer or evaluator has no way to tell which numbers on the screen are real and which are invented. This must be fixed before anyone outside the team looks at it closely.

This document works through all 21 requested phases. Given the size of the input (a ~5,000-word SRS and a ~30-file implementation), depth is allocated where it changes decisions — the product audit, the ML audit, the UI/data-integrity audit, the database audit, the SRS audit, and the final blueprint/roadmap get full treatment; sections where the SRS is already reasonable (testing strategy, backup/recovery, performance targets) get a shorter, direct pass rather than restating what's already adequate.

PART 1 — CURRENT SYSTEM RECONSTRUCTION

What actually exists (verified by inspection, not by the docs' claims about themselves):

Layer	Reality
Frontend	React 18 + Vite. No router library (hand-rolled hash router in `RouterContext.jsx`), no chart library (hand-rolled inline SVG in `Charts.jsx`), no CSS framework, no state library. Two runtime dependencies total.
Backend	**None.** No FastAPI, no server, no API calls anywhere in the codebase.
Database	**None.** `data/synthetic/*.csv` (13 files, 200 students) are compiled by `scripts/build_prototype_data.py` into a single static `src/data/syntheticData.json`, imported directly into the bundle.
Auth	`AuthContext.jsx` holds a `currentUser` in React state. `loginWithUsername(username, password)` looks up a user by username only — **the password argument is never read or checked.** A "Simulate Role" dropdown lets anyone become any of the 9 personas in one click.
RBAC	A hardcoded `ROLE_SCREEN_PERMISSIONS` object gates which screen components render. This is client-side conditional rendering, not authorization — correctly and explicitly labeled as such in `Forbidden403.jsx` and the prototype-review doc.
"ML"	`PREDICTION_CASES` is a JS object with exactly **two** hardcoded students (one "high risk," one "low risk"), including invented SHAP-style factor weights and an invented `precision: 84.2%` / `recall: 81.5%` for a model that has never been trained, on data that does not exist yet. Labeled `SIMULATED / PROTOTYPE SPECIFICATION ONLY` in the metadata object and on-screen.
Data volume	200 students, 3 departments, 36 subjects, 4,278 marks rows, 1,426 attendance rows, 648 semester-result rows, 52 backlogs. All students are a **single admitted cohort (2024–2028), currently in Semester 3.** There is no second cohort with a completed academic history.
Docs	10 markdown documents (6,234 lines total) covering SRS, ERD, data dictionary, database design + a self-review, synthetic data design, coding standards, UI wireframes, role-permission matrix, and a prototype conformance audit.

What is simulated vs. real vs. proposed vs. future-semester, cutting across the whole system:

- **Real:** the relational structure of the synthetic CSVs, the SGPA/attendance/backlog aggregation logic for student- and mentor-level views, the routing/RBAC-gating logic, the build tooling.
- **Simulated (labeled):** login/session, RBAC enforcement, ML model + predictions, system telemetry (SCR-016), file export/PDF generation, audit log persistence.

- **Simulated (unlabeled — this is the problem):** several "analytics" numbers on SCR-002 and SCR-008 that are hardcoded literals dressed up as computed metrics, indistinguishable on screen from the real ones next to them.
- **Proposed, not SRS-mandated:** the 1-click role switcher, the persona picker on login, the mentee/student selector widgets. These are reasonable prototype conveniences, correctly flagged as such in the docs.
- **Future-semester, correctly deferred:** FastAPI/PostgreSQL runtime, JWT/bcrypt auth, Celery/Redis workers, real scikit-learn training, SHAP, email/SMS delivery.

PART 2 — SRS TRUTH TABLE

A representative sample (not exhaustive — the full SRS has ~50 functional requirements across 16 modules); classified per the requested A–J scheme.

Feature / Requirement	Class	Note
Role-based dashboards for 9 roles	A (explicit, SRS §10)	Implemented as 16 screens gated by role
SGPA/CGPA/attendance/backlog tracking	A (explicit, SRS §9, §20)	Computed correctly from real synthetic data where not hardcoded
Controlled two-stage data import (validate → commit)	A (explicit, SRS §19, BR-005/006)	Simulated with one canned job (`IMP-104`) with two seeded errors
ML prediction ("optional… when sufficient authorized data exists")	B (explicit but conditional, SRS §21.1)	**Implemented ahead of its own precondition** — a fully-specified model name, version, and metrics exist despite the SRS's own eligibility gate not being met
75% attendance threshold	G→resolved (ambiguous in SRS, correctly resolved by the team)	SRS never mandates a number; team correctly relabeled it "Configurable Baseline (Design Assumption)"
SHAP explainability	G→resolved	SRS says "explanations… where feasible," does not name SHAP; team correctly walked this back to "conceptual feature attribution"
`RolePermissions` junction table	G, self-identified by team	SRS lists `Roles` and `Permissions` with no join table — team's own `database-review.md` flags this, correctly, as unresolved
Faculty→Subject→Section teaching allocation	H (technically questionable)	BR-003 requires faculty scope enforcement; no entity in the 27-table schema can represent "which faculty teaches which subject to which batch," so BR-003 is currently **unenforceable at the data layer**, not just unbuilt
Curriculum (Program→Semester→Subject) mapping	H	Same root cause — `Subjects` has no FK to `Programs`/`Semesters`; a subject is only reachable through an `Assessment`, so nothing prevents "curriculum" being an

Feature / Requirement	Class	Note
		emergent side-effect of whichever assessments were entered rather than an authoritative structure
`MentorNotes` persistence	D (added during implementation, not in SRS or ERD)	UI feature exists (SCR-006 "record mentoring note"); implementation stores it in an in-memory JS object with no corresponding database entity anywhere in the 27-table design
Precision/Recall/F1 for `AcademicRiskClassifier`	I (product-wise questionable), verging on J	Presented as a "Target/Benchmark" in a table row that looks identical to a real reported metric. No model exists; the numbers are invented and unfalsifiable at this stage
Prototype 1-click role switcher	D	Not in SRS; reasonable evaluator convenience, clearly labeled
Department drill-down into individual student profile	G, resolved by team	SRS doesn't specify; team chose to block it for privacy (BR-004) — a defensible, conservative default
System health telemetry screen (SCR-016) for a system with no server	C, technically premature	Logically derivable from SRS §26/§27, but building the *screen* before there is any real system to monitor inverts the natural build order — this is more Semester-8 UI than Semester-3 UI

PART 3 — PRODUCT PROBLEM AUDIT (the uncomfortable part)

Answering the SRS's own framing directly:

1. **What real problem is SAAP solving?** As specified: fragmented academic data (marks, attendance, results, backlogs) living in disconnected spreadsheets/registers, making it hard to see a student's trajectory or a cohort's weak spots in one place.
2. **Who has the problem?** Mentors and department heads, primarily — they are the ones currently reconstructing "who needs help" by hand from multiple sources.
3. **How serious is the problem?** Real but modest. It is an information-friction problem, not a crisis. Institutions survive without SAAP today; they just do it slower and less

consistently.

4. **What happens today without SAAP?** A mentor manually checks a spreadsheet or asks a student directly. It works, imperfectly, at small scale (dozens of mentees).
5. **What workflow does SAAP improve?** In the current design: the *finding* half of "find a struggling student." It does nothing yet for the *doing something about it* half.
6. **What decisions does SAAP help people make?** "Who should I check on?" That's genuinely useful. It currently does **not** help anyone decide what intervention to try, whether a past intervention worked, or when to escalate.
7. **What actions happen because of SAAP?** None are modeled. A mentor can view an at-risk student and add a free-text note. There is no assignment, no due date, no escalation, no record of what was tried and whether it worked.
8. **What measurable outcome could improve?** Time-to-detection of a struggling student, and (if a follow-up loop existed) intervention-to-recovery rate. Right now only the first is even partially addressed.
9. **Why does this need to exist as a separate platform, rather than a dashboard inside the existing ERP?** Honestly — on the current scope, it doesn't clearly need to. Everything implemented so far (marks, attendance, SGPA, backlog views, aggregate charts) is exactly what a "reports" module bolted onto an ERP already does at most institutions. The differentiator SAAP is *reaching for* — decision support, intervention tracking, explainable risk — is precisely the part that isn't built yet.
10. **Why isn't this simply a dashboard inside an ERP?** It could be, today. The justification for a standalone platform will only exist once there's a genuine workflow layer (case management, intervention tracking, outcome feedback) that a bolt-on ERP report can't easily provide.
11. **Actual technical contribution today:** a clean, dependency-light, role-scoped read-only reporting layer over academic data. Competent, not novel.
12. **Actual product contribution today:** none yet beyond "the reports are prettier and role-scoped." That's a legitimate incremental improvement, not a platform.
13. **What is genuinely interesting?** The insight-generation concept (SCR-013) *if* it were wired to a real action loop. The two-stage controlled import with a pre-commit diagnostic screen (SCR-009/010) is also a solid, under-appreciated piece of real institutional software design — most student ERP clones skip validation entirely and this team didn't.
14. **What is generic?** The 16-screen, one-dashboard-per-role structure; the SGPA/CGPA/attendance metric cards; the "Insights & Recommendations" screen that is really just a filtered list of the same underlying facts restated as sentences.
15. **What is unnecessary, at this stage?** SCR-016 (system health) for a system that has no backend to monitor yet; SCR-012 (user management) in full CRUD form before there's a real user store to manage.

16. **What feels like "AI for the sake of AI"?** The `AcademicRiskClassifier` model card with a named algorithm (Random Forest), a version number (v1.2.0), and invented precision/recall — for a model that, per the SRS's own eligibility rule, should not exist yet because the historical data doesn't exist yet either. This is the clearest single instance of the platform performing sophistication it hasn't earned.
17. **What feels like "dashboard for the sake of dashboard"?** SCR-007 Faculty Analytics and SCR-008 Department Analytics largely restate SCR-002/006 at a different aggregation level with different chart shapes. There is no faculty- or department-specific *decision* these screens uniquely support yet.

Bottom line for Part 3: SAAP today is a competent, honestly-built **academic reporting layer**, wearing the branding of a **decision-support / AI platform** it has not yet built the workflow for. That branding gap is fixable, but only by product redesign (Part 5), not by writing a backend.

PART 4 — REAL-WORLD PLATFORM PATTERN ANALYSIS

(Documented external fact vs. architectural inference is marked; nothing below is presented as a specific competitor's confirmed feature list — treat this as pattern-level, not feature-level, comparison.)

Institutional "student success" and early-alert software (a well-established category in higher-ed IT, generally sitting alongside SIS/LMS platforms) is built around a consistent pattern, independent of vendor: **flag → case → action → outcome.** A flag (a risk score, a threshold breach, a faculty-raised concern) creates a *case* owned by a specific person (advisor/mentor), who takes a *logged action* (referral, meeting, resource assignment), and the case is later closed with an *outcome* that feeds back into whether the flagging logic is any good. *(Architectural inference, drawn from how case-management and CRM-adjacent systems are conventionally structured — not a claim about any named product's internals.)*

What SAAP should borrow from this pattern (recommendation, not an SRS requirement):

- A **case/intervention entity** with a status lifecycle (open → in progress → resolved/escalated), owned by a mentor or advisor, linked to the insight or risk signal that triggered it.
- **Role separation between "who sees a signal" and "who owns acting on it"** — currently SAAP shows the same insight to student, mentor, and department user with no ownership concept.
- **Explainability as evidence for a human, not a black box grade** — SAAP's instinct to show "supporting factors" is correct; it just needs to point at a *case*, not a floating card.

What SAAP should explicitly NOT copy:

- Vanity dashboard density (many cards, few decisions) — common in generic BI tooling and already a weakness in SAAP's current screens (Part 6 below).
- Automating any high-stakes academic decision (probation, deregistration) — the SRS is correct to keep this out (BR-009, "predictive indicators do not replace human evaluation") and should keep it out permanently, not just at the prototype stage.
- Building a general-purpose BI/self-service query tool — that's a different product (and a much bigger one) than a student-success platform; SAAP's current SRS wisely doesn't ask for this, and shouldn't drift toward it under "analytics" scope creep.

PART 5 — CURRENT PRODUCT CRITIQUE / REDESIGN

Do not keep the current 16-screens-because-16-screens-were-speced structure by default. The right redesign question is: what is the loop, and what screens does the loop actually need?

Recommended core loop (not the SRS's DATA→…→INTELLIGENCE vision line, which is a *maturity* arc, not a *workflow* loop):

```
SIGNAL (threshold/insight fires)
   → REVIEW (mentor/faculty sees evidence, not just a score)
   → CASE (someone takes ownership, logs an intended action)
   → FOLLOW-UP (was it done? did anything change?)
   → OUTCOME (resolved / escalated / no change) → feeds back into which signals are
worth trusting
```

Primary users: Mentor, Faculty (subject-level), Academic Administrator. **Secondary users:** Student (recipient of insights, not a workflow actor), Department User, Data Analyst/ML Engineer (internal, model governance), System Administrator/Auditor (internal, compliance).

Screen-level redesign implications (detail in Part 6):

- Merge **SCR-013 (Insights)** into the case workflow — an insight without a "turn this into a case" action is a dead end today.
- SCR-014 (Prediction Analytics) should not be a standalone screen users browse; a risk score belongs *inside* the case review, as evidence, not as its own destination.
- SCR-006 (Mentor Dashboard) is closest to correct already — it has a roster, a risk flag, and a notes field. It needs the notes field upgraded to an actual case with a status, not free text that vanishes on reload.
- SCR-016 should not exist as a full standalone screen at Semester 3; fold a minimal "recent activity" view into SCR-012 until there's a real backend worth monitoring.

PART 6 — UI/UX AUDIT (representative screens)

Full screen-by-screen critique of all 16 screens would roughly double this document's length; the pattern repeats, so three representative screens are audited in full depth and the rest are summarized in the table that follows.

SCR-002 — Student Dashboard (full audit)

1. **Purpose:** Give a student a one-glance view of standing.
2. **Current implementation:** Real SGPA/CGPA/attendance computed from `dataService`, plus a bar chart (`subjectBars`) and a monthly attendance trend (`monthlyAttPoints`) that are **hardcoded arrays with no connection to the logged-in student's actual data**, plus a "Cohort Rank: Top 15%" and "+0.25 trend" footnote that are **static strings**, unrelated to whichever student is actually logged in.
3. **SRS requirement:** SRS §17 asks for a student dashboard with KPIs and trends; it does not ask for fabricated percentile ranks.
4. **What's good:** Clean KPI-card layout, correct real computation for the primary three metrics, sensible click-through to detail screens.
5. **What's weak:** The fabricated numbers are placed in the exact same visual weight as the real ones. A viewer cannot tell "88 marks in CS301" (invented) from "SGPA 8.42" (computed) without reading the source.
6. **What's generic:** Four-metric-card-grid-plus-two-charts is the default shape of every dashboard template; nothing here is dashboard-specific to academic advising.
7. **What looks AI-generated / template-like:** The bar chart's "tags" (`High`, `Near Min`, `Stable`) read like placeholder labels a generator would produce to look sophisticated without being wired to real thresholds.
8. **What's unnecessary:** The cohort-percentile footnote, as currently implemented, is worse than absent — remove it or compute it for real.
9. **Information hierarchy problem:** "Download Transcript" (a rarely-used action) gets equal button prominence to "View Official Profile" (a frequently-used one).
10. **Missing workflow:** No link from "attendance near threshold" to any action the student can take (e.g., request a mentor check-in).
11. **Remove:** hardcoded subject bars, hardcoded monthly trend, hardcoded percentile.
12. **Add:** a real per-subject rollup computed the same way the mentee roster already computes attendance (the pattern exists elsewhere in `dataService.js` — it just wasn't reused here).
13. **Redesign:** Replace the decorative bar chart with the *actual* subject-wise marks already available via `getStudentMarks()`.

14. **Recommended final structure:** 3 real KPI cards, 1 real SGPA/CGPA trend (already computed correctly elsewhere in this same file), 1 real subject-wise table, 1 real attendance-by-subject list, insights feed at the bottom. No invented numbers anywhere.

SCR-014 — Prediction Analytics (full audit)

1. **Purpose:** Show risk scores with supporting evidence to mentors/faculty.
2. **Current implementation:** Exactly 2 hardcoded students, invented SHAP-style weights, invented model metrics.
3. **SRS requirement:** §21 explicitly gates any prediction work behind data sufficiency review — this screen exists ahead of that gate.
4. **What's good:** The "supporting factors" concept and the disclaimer banner (BR-009 language) are the right instincts.
5. **What's weak:** It looks and reads like a real model output. A department head or accreditation reviewer skimming this screen has no reason to suspect it's two hand-picked examples.
6. **What's generic:** Risk gauge + factor list is the default shape of every "explainable AI" mockup.
7. **AI-generated feel:** The precision/recall/F1 numbers, specifically — round, plausible-looking, and entirely fictional. This is the single highest-risk item in the whole prototype if shown to an evaluator who takes it at face value.
8. **Unnecessary at this phase:** the entire screen, as a destination. A model that doesn't exist yet doesn't need a governance UI yet.
9. **What's missing:** Nothing — the problem is that too much exists, prematurely.
10. **Should be removed:** the standalone screen. **Should be redesigned:** as a small "Planned: Predictive Risk Indicators (Semester 6, pending data sufficiency review)" placeholder with the SRS's own eligibility criteria shown as a checklist against real data — which, per Part 11 below, is not yet met and should be shown as not yet met, not simulated as if it were.

SCR-016 — Audit & System Health (summary audit)

Purpose and intent are fine (SRS §26/27); execution is premature — full telemetry cards for a database and API that don't exist yet invite exactly the "is this real?" confusion the team otherwise worked hard to avoid. Recommend folding this into a smaller "Recent Activity" panel until Semester 4 produces a real backend to actually monitor.

Remaining screens — summary table

Screen	Verdict
SCR-001 Login	Fine for prototype phase; the persona picker should visibly disappear once real auth exists — flag this now so it isn't forgotten in Sem 4.
SCR-003 Academic Profile	Solid, most "ERP-like" screen; correctly scoped to real data.
SCR-004/005 Marks/Attendance	Real computation, reasonable role variants (student view vs. faculty entry view). Keep.
SCR-006 Mentor Dashboard	Best screen in the set — closest to the recommended case-workflow shape. Upgrade notes to persisted cases (Part 5).
SCR-007 Faculty Analytics	Generic restatement of department-level data at subject grain; needs a faculty-specific decision it uniquely supports (e.g., "which of my sections needs a remedial session") or should be merged into SCR-008 with a filter.
SCR-008 Department Analytics	Real average-marks computation sits next to hardcoded pass percentages (`'76.2%' : '92.5%' : '88.0%'` by string match) — same fabrication problem as SCR-002. Fix before anything else on this screen.
SCR-009/010 Import Center/Validation	Genuinely good design for a student project — two-stage validate/commit is real institutional practice. Keep the shape; the current single canned job is fine for Sem 3.
SCR-011/012 Master Data/User Mgmt	Reasonable CRUD scaffolding; low priority to polish further until Sem 4 backend exists.
SCR-013 Insights	See Part 5 — needs a "convert to case" action or it's a dead-end list.
SCR-015 Reports	Fine as a job-queue simulation; keep.

PART 7 — LANDING / ENTRY EXPERIENCE AUDIT

An institutional academic platform does **not** need a marketing landing page, and the SRS correctly never asks for one. What it needs, and mostly has:

- **Institutional login, not a consumer sign-up flow** — correct, present.
- **Role-appropriate landing route per persona** (`ROLE_DEFAULT_ROUTES`) — correct, present, and a genuinely nice detail.

- **What should NOT be visible pre-auth:** any real student data, any real risk scores — currently satisfied since nothing real exists yet, but this constraint should be written down now as a requirement for Semester 4, not rediscovered later.
- **What's currently over-exposed for a "production-shaped" prototype:** the 1-click "become any of 9 roles" picker sitting directly on the login screen. Fine for an evaluator demo; must not survive into anything resembling a real deployment, and should be explicitly labeled in the SRS roadmap as "prototype-only, removed in Semester 4," not left implicit.
- **Trust/security cues:** none needed yet at prototype phase; will matter once real student PII is involved (institution branding, session-timeout notice, data-handling notice).

PART 8 — TECHNICAL ARCHITECTURE AUDIT

Frontend: Appropriate and disciplined for the phase. No router library, no chart library, no CSS framework — a deliberate minimalism the team documented and justified (`prototype-review.md §6/7`), which is the correct call at Semester 3 and should **not** be treated as a permanent constraint — a real router (React Router) and a real chart library become worth their weight once there's server-driven pagination and more than a handful of chart types.

Backend (proposed, SRS §11/14): FastAPI is a reasonable, appropriately-scoped choice for this project's size — not over-engineered, not under-powered. The layered proposal (API → business services → data access) is standard and fine; no changes recommended to the *shape*, only to sequencing (see Part 19 — don't build SCR-016-grade telemetry before there's a service worth monitoring).

Database: PostgreSQL is the right choice and is not being second-guessed here. See Part 9 for schema-level findings.

Auth: The SRS's plan (JWT + bcrypt + RBAC) is standard and sufficient for this scope; no over- or under-engineering to flag. One planning note: the *current* prototype's password-blind login (Part 1) should be explicitly called out in the Sem 4 handoff notes so it isn't accidentally carried forward as a "shortcut that already works."

Infrastructure: SRS's Docker/Linux deployment plan is appropriately modest — no criticism; a Semester 3–8 CSE project does not need Kubernetes, a message queue beyond Celery/Redis, or multi-region anything, and the SRS correctly doesn't ask for any of that.

PART 9 — DATABASE AUDIT

The team's own `database-review.md` already surfaces most of the real issues (missing `RolePermissions`, missing curriculum mapping, missing faculty-teaching-allocation, nullable-

FK ambiguities) — that self-review is accurate and should be trusted rather than redone from scratch. This section adds what wasn't in it:

Missing relative to what the *implementation* actually needs, that the schema review didn't flag:

- `MentorNotes` / **case entity** — the prototype code already invents an in-memory notes store (`mentoringNotes[studentId]`) that has zero representation in the 27-table ERD. This is a real gap between "what the app does" and "what the database can persist," not a hypothetical one.
- **An** `InterventionCases` **entity** — needed if Part 5's redesign is adopted; does not exist in the current 27 tables at all, because the current design has no case-management concept to model.

Current database: 27 tables, correctly normalized in most modules (assessments/marks/attendance/results), UUID strategy well-justified, audit logging present.

Problems:

- No `Roles` ↔ `Permissions` join table → permissions are effectively hardcoded in frontend JS today (`ROLE_SCREEN_PERMISSIONS`), which cannot survive a real backend unchanged.
- No `Subjects` → `Programs` / `Semesters` FK → curriculum is not an authoritative structure, only an emergent one.
- No faculty-to-subject-to-section allocation table → BR-003 (faculty scoped access) is currently a business rule with no data structure to enforce it against.
- No entity for mentoring notes/cases, despite the UI already implying one exists.

Required changes: add `RolePermissions`, add `CurriculumMap` (Program×Semester×Subject), add `TeachingAllocations` (Faculty×Subject×Batch×Semester), add `InterventionCases` (if Part 5 is adopted) or at minimum `MentorNotes` (if it is not).

Final recommendation: 27 → 31 tables (or 32 with `InterventionCases`), all additive, none of the existing 27 need to be removed or restructured. This is a small, low-risk change set — worth doing before Semester 4 migrations are written, not after.

PART 10 — DATA ARCHITECTURE AUDIT

The synthetic dataset is realistic in *shape* (proper foreign-keyed CSVs, sensible row-to-entity ratios: 4,278 marks / 200 students / 36 subjects ≈ 21 marks per student, which is plausible for ~4 subjects × ~3 assessment types × ~2 semesters completed).

What it cannot support, structurally, regardless of volume: every one of the 200 students belongs to a *single admission cohort* (2024–2028), currently in Semester 3. There is no

completed cohort, no graduated students, no confirmed dropouts, no multi-year longitudinal record. This is not a "not enough rows" problem — it's a "the outcome variable doesn't exist in the data yet" problem, and it's structural, not something more synthetic-data generation fixes on its own. This single fact should drive the entire ML section (Part 12).

PART 11 — ANALYTICS AUDIT

Running the SRS's own metric list through "who needs it / what decision follows":

Metric	Decision it should support	Currently meaningful?
SGPA/CGPA trend	Student self-monitoring; mentor triage	Yes — real, computed, useful as-is
Attendance % + trend	"Am I at risk of exam ineligibility"	Yes — real, and the threshold-as-configurable-assumption framing is correct
Backlog count/pattern	Curriculum weak-point identification (e.g., CS302)	Yes — real, and genuinely the most actionable metric in the whole system
Cohort/percentile rank	Motivational framing for the student	**No** — currently fabricated, not computed at all; either compute it for real (cheap — it's a sort over `semester_results`) or remove it
Department pass rate	Curriculum/faculty review	**Currently fabricated by string match, not computed** — must be fixed to compute from `student_marks`/`semester_results` the same way the average-marks figure next to it already is
"Risk flag" (Normal/Attendance/Marks Drop/Backlog)	Mentor triage priority	Yes — simple, real, reproducible threshold logic; this is a good example of a *rule-based* insight done right

Recommendation: every metric that is currently real should stay; every metric that is currently fabricated should either be computed for real (most of them can be, cheaply, using logic that already exists elsewhere in the same file) or removed. Do not add new metrics until this cleanup is done — more charts is not the goal here (SRS Rule 6, correctly stated in the prompt).

PART 12 — AI/ML AUDIT (the most important section)

Walking the SRS's own §21.3 questions against the actual data (Part 10):

1. **Which actual problem requires prediction?** "Which currently-enrolled student is likely to accumulate a new backlog this semester" is the only candidate the data could plausibly support later — and even that needs at least one full semester's worth of *outcome* (pass/fail) per predictor-period, which the single in-progress cohort doesn't yet have.
2. **Target variable?** Undefined in the current data — there is no completed "did this student get a backlog" outcome tied to a prior-period feature snapshot yet, because the cohort hasn't finished a semester with a graded outcome that could be paired with an *earlier* snapshot.
3. **Historical data required?** At minimum, one full cohort's worth of semester-over-semester (features-at-time-T → outcome-at-T+1) records — a repeat of the current single-cohort dataset across 2+ more completed semesters, ideally 2+ cohorts for any hope of generalization.
4. **Features available?** Marks, attendance, backlog history — reasonable feature candidates, structurally present in the schema.
5. **Prediction horizon?** Not yet specified anywhere in the SRS — should be (e.g., "risk of new backlog by end of current semester") before any model work starts.
6. **Baseline?** None defined. The SRS should require a trivial baseline (e.g., "flag anyone below 75% attendance OR with an active backlog") be benchmarked *before* any ML model is allowed to claim value over it — this is missing from §21 and should be added.
7. **Metrics?** Precision/recall/F1 are reasonable choices for a rare-event classifier — but the specific numbers currently in the prototype are invented and must not survive into Semester 6 planning documents even as "targets," because a fabricated target risks being mistaken for a validated one.
8. **Leakage risks?** High, given the feature set — e.g., using "this semester's SGPA" to predict "this semester's backlog" is circular; the SRS doesn't yet discuss this and should before Semester 6.
9. **Validation?** Needs a temporal split (train on earlier cohort/semesters, validate on later ones), not a random split — random splitting on this kind of data silently leaks information across a student's own repeated records.
10. **Uncertainty representation?** Not addressed yet; reasonable to defer to Semester 6.
11. **Explanations?** The instinct (simple factor list, not raw SHAP) is right; defer the specific method until real data exists to check whether a tree-based model's default importances are even worth explaining, versus a simpler and more explainable logistic/rule-based model given the small likely dataset size.

12. **Decisions humans can make using it?** "Prioritize this student for a mentor check-in this week" — appropriately human-in-the-loop, matches BR-009.
13. **Decisions that must NOT be automated?** Anything with academic consequence (grade adjustment, probation, deregistration) — SRS already excludes this correctly; keep excluding it permanently.

Direct instruction the SRS itself would give if followed literally: do not build a classifier yet. The data sufficiency and quality review that SRS §21.1 requires *before* any ML work has not happened, cannot honestly pass today (Part 10), and the prototype has nonetheless produced a named model with a version number and invented accuracy figures. This is the single highest-priority fix in this entire review: **remove the specific model name/version/metrics from the prototype and documentation until Semester 6's own data-sufficiency review can actually be run for real.** Until then, if a "prediction-shaped" screen is wanted at all, it should be a rule-based risk flag (which already exists correctly elsewhere in `dataService.js`) explicitly labeled as rules, not ML.

PART 13 — SIMULATION AUDIT

Simulation	Represents	Convincing?	False impression risk	Recommendation
Client-side login	JWT/session auth	Yes, and explicitly disclaimed on-screen	Low — well labeled	Keep for Sem 3; remove persona picker in Sem 4
Client-side RBAC	Server-side authorization	Yes, and explicitly disclaimed in `Forbidden403`	Low — well labeled	Keep for Sem 3
System health telemetry	Real PostgreSQL/API metrics	Yes, and explicitly disclaimed	Low — well labeled, but see Part 6 (premature screen)	Shrink or defer
ML model metadata + predictions	A trained classifier	Yes — dangerously so	**High** — labeled "SIMULATED" in one place but presented with specific, plausible-looking metrics elsewhere	Remove specific numbers; keep only the concept mock
SCR-002/SCR-008 fabricated metrics	Real computed analytics	Yes — **not labeled at all**	**Highest in the system**	Fix immediately — either compute for real or remove
Import validation (`IMP-104`)	Real ingestion pipeline	Yes, and explicitly framed as one seeded example	Low	Keep

The pattern across this table is consistent: **everything the team consciously decided to simulate, they labeled well.** The two dangerous items (ML metrics, SCR-002/008 hardcoded numbers) are dangerous specifically because they weren't treated as "the kind of thing that needs a disclaimer" — they slipped in as ordinary-looking UI content. That's a process fix (a "no literal numbers outside `dataService.js`" rule), not a talent fix.

PART 14 — CODE AUDIT

Issue	Severity	Fix before Sem 4?
Hardcoded fabricated metrics in SCR-002, SCR-008	**CRITICAL**	Yes — before showing this to anyone external
Invented ML precision/recall/F1 presented as benchmarks	**CRITICAL**	Yes — remove from code and docs
`loginWithUsername` never checks the password argument	MEDIUM	No — acceptable for a declared client-side simulation, but note it explicitly so it isn't silently carried into Sem 4 auth work
In-memory `mentoringNotes` / `auditLogs` / `importJobs` mutate module-level `let` variables (not React state)	LOW	No — fine for a static prototype; would need to change once there's a real backend anyway
No automated tests anywhere in the repo	MEDIUM	No — acceptable at Semester 3 (SRS's own testing plan starts later), but shouldn't slip past Semester 4
`getDepartmentSummary` mixes real aggregation with ternary-hardcoded values in the same return object	**HIGH**	Yes — this is the SCR-008 fabrication issue at its source
Minimal dependency footprint (2 runtime deps)	Not an issue — a strength	—
Every file has an active consumer (team's own QA check verified)	Not an issue — confirmed accurate on inspection	—

Can remain because it's only a Semester-3 prototype: volatile in-memory state, absence of a router/chart library, absence of tests, absence of a backend. **Must fix before Semester 4:** both hardcoded-fabrication issues above — these are the only two findings in this entire review classified CRITICAL, and both are small, mechanical fixes (either wire the existing correct computation logic through, or delete the literal).

PART 15 — SRS AUDIT

SRS Section	Requirement	Problem	Severity	Recommended change
§21.1	ML "shall only proceed after sufficient… data is available"	Correct rule, but the prototype (§9 features list implies ML is a current feature) doesn't visibly gate on it — the SRS doesn't say *where* or *how* this gate is checked/recorded	MEDIUM	Add an explicit "Data Sufficiency Review" deliverable/checkpoint before Semester 6, with a named artifact (a short report), not just a principle
§18	27 database entities	No `RolePermissions`, curriculum mapping, or teaching-allocation entities, despite dependent business rules (BR-003) requiring them	HIGH	Add the 3–4 tables identified in Part 9
§9	"Optional ML prediction" listed as a current-tier feature alongside CRUD features	Reads as same-priority as core features; a reader skimming §9 would not guess ML is conditional and years away	LOW	Move the ML bullet out of §9 into its own conditionally-worded line, or add a footnote pointing to §21.1
§26	Dashboard response time targets	Reasonable, not contradicted by anything found	—	No change needed
§24 (BR-003)	"Faculty access restricted to authorized academic scope"	No database entity can currently represent "scope" for a faculty member (Part 9)	HIGH	Either add the entity (recommended) or explicitly scope BR-003 down to something the current schema *can* enforce
Appendix A	27 entities "exactly as specified"	Encourages treating the count as sacred; the team's own review correctly proposes additions but frames them as "needs	LOW	Reframe Appendix A as a floor, not a ceiling

SRS Section	Requirement	Problem	Severity	Recommended change
		confirmation" rather than "will be added"		
§20	"Insight statements should link to supporting metrics"	Present in spirit (SCR-013 cards do cite evidence), but no requirement that an insight can be *acted on* (Part 5)	MEDIUM	Add a requirement for an intervention/case linkage if Part 5's redesign is adopted

No contradictions, duplicated requirements, or genuinely unrealistic asks were found in the SRS body — it is a measured, appropriately-scoped document. The issues above are gaps and sequencing risks, not internal inconsistencies.

PART 16 — KEEP / CHANGE / DELETE

KEEP

- The two-stage controlled import concept (SCR-009/010) and its underlying `ImportJobs`/`ImportErrors` schema.
- The role→default-route mapping and role-scoped sidebar.
- The dependency-minimal frontend approach for this phase.
- The rule-based risk-flag logic in `getMentorMentees` (real, simple, reproducible — the best piece of "analytics" in the app).
- The SRS's conservative ML-eligibility language and human-in-the-loop stance (BR-009).
- The team's own self-audit habit (`prototype-review.md §14`) — this should become a standing practice each semester, not a one-off.

CHANGE

- The AI/ML story: from "named model with invented metrics" to "eligibility checklist, not yet met" (Part 12).
- The Insights screen: from a dead-end list to a case-creation trigger (Part 5).
- SCR-007/008: from generic restatements to screens tied to a specific decision each role uniquely makes.
- The database: add the 3–4 missing entities (Part 9) before Semester 4 migrations.
- SCR-016: shrink until there's a real system to monitor.

DELETE

- The hardcoded fabricated metrics in SCR-002 and SCR-008 (or convert them to real computations — functionally a "delete the literal, wire the real logic" operation).
- The specific invented precision/recall/F1 figures, wherever they currently appear (code and any documentation echoing them).
- The full-scale SCR-014 as a standalone destination, at this phase (replace with the eligibility-checklist placeholder above).

PART 17 — FINAL PRODUCT BLUEPRINT

1. **Product vision (revised):** SAAP closes the loop from *academic signal* to *mentor action* to *tracked outcome* for a defined cohort — not "AI-powered academic intelligence," which is not yet true and shouldn't be claimed until it is.
2. **Core problem:** Mentors and department staff lack a single, role-scoped place to see who needs attention *and* to track whether they followed up.
3. **Target users:** Mentors and Academic Administrators (primary); Faculty and Department Users (secondary); Students (recipients, not workflow actors); Data Analyst/ML Engineer/System Administrator/Auditor (internal governance roles, correctly present in the SRS already).
4. **Core value proposition:** One place to see academic risk signals *and* close the loop on what was done about them — the second half is what a bolt-on ERP report can't easily do, and is the actual justification for a standalone platform (Part 3, Q9).
5. **Core workflow:** SIGNAL → REVIEW → CASE → FOLLOW-UP → OUTCOME (Part 5).
6. **Feature hierarchy:** (1) accurate role-scoped reporting [mostly done], (2) rule-based risk flags [done, keep], (3) case/intervention tracking [not yet built — highest-value net-new feature], (4) explainable ML risk scoring [correctly deferred to Sem 6, pending real eligibility].
7. **Information architecture:** collapse SCR-007/008 toward decision-specific views rather than generic aggregate restatements; fold SCR-013/014 into the case workflow rather than standalone browsing screens.
8. **Final screen map:** see Appendix A.
9. **Role model:** the existing 9 roles are appropriate; no additions or removals recommended.
10. **Permission model:** add the `RolePermissions` table (Part 9) so permissions become data, not hardcoded frontend JS, ahead of Semester 4. 11–19: covered in Parts 8, 9, 11, 12, 13 respectively (backend, DB, analytics, ML, audit/security architecture); reporting and import architecture are already fit-for-purpose as designed (Part 6) and need no structural change, only the data-integrity fixes in Part 14.
11. **Notification/intervention architecture:** justified — this is the actual missing piece identified throughout this review (Part 3, 5, 17.6). Minimal version: a case gets a status

and an owner; a due-date reminder is a reasonable Semester 5/7 addition, not a Semester 4 one. 21–23: see Part 19 (roadmap).

PART 18 — FINAL ARCHITECTURE (delta from SRS, not a rewrite)

The SRS's proposed architecture (React → FastAPI → PostgreSQL, layered business services, optional ML layer) is **not being replaced** — it's sound for this project's scope. The only deltas recommended by this review:

- **Database:** +3–4 tables (Part 9).
- **API surface:** SRS's endpoint list (§22) needs one addition — an `interventions`/`cases` resource — if Part 5/17 is adopted; otherwise no change.
- **AI layer:** demote from "a named model exists" to "an eligibility-gate check exists," until Semester 6's real data-sufficiency review can run (Part 12).
- **Permission model:** move from hardcoded frontend map to a real `RolePermissions` table, consumed by both frontend and backend, so the two can't drift apart the way a hardcoded JS object inevitably will.

PART 19 — DEVELOPMENT ROADMAP (incremental from current state)

Semester 4 — Core Web Application *(as SRS §30 already plans, plus:)*

- Build the +3–4 database tables identified in Part 9 into the initial migrations, not as a later patch.
- Fix the SCR-002/SCR-008 fabrication issues as part of porting these screens to real API calls (they need to be touched anyway to wire to a backend — do the honesty fix at the same time, not separately).
- Do **not** yet build a cases/interventions API — CRUD + auth + RBAC is enough scope for one semester.
- Must NOT build: any ML inference code, any SHAP/explainability code — still gated by Semester 6.

Semester 5 — Analytics & Visualization *(as planned, plus:)*

- Build the `InterventionCases` entity and minimal case-status API (open/in-progress/resolved) — this is the natural home for it, since it's a business-logic feature, not a data-pipeline one.
- Re-implement the currently-fabricated department pass-rate and student percentile metrics as real, server-computed aggregates.

Semester 6 — ML Exploration *(as planned, with one addition:)*

- **Run the data-sufficiency review for real, and publish its result even if the answer is "not yet."** By this point the cohort will have completed 2–3 more semesters, which meaningfully improves the odds the eligibility bar can actually be met — but it must be checked, not assumed.
- If eligibility is met: build a baseline (simple threshold rule) and a candidate model, and report both — never present the model's numbers without the baseline for comparison (fixes the Part 12 root issue at the process level, not just the data level).
- If eligibility is not met: explicitly document why, and defer — do not ship a "simulated-but-specific" model card again.

Semester 7 — Recommendations & Explainable Analytics: as planned; now has a real case workflow to plug recommendations into, rather than a floating insights list.

Semester 8 — Quality, Security, Deployment: as planned; no changes recommended.

PART 20 — PRIORITY MATRIX

Priority	Problem	Why	Action	Phase
P0	Hardcoded fabricated metrics in SCR-002/SCR-008	Undermines trust in every real number on the same screen	Compute for real or delete	Immediately (before further Sem 3 review/demo)
P0	Invented ML precision/recall/F1 for a nonexistent model	Directly contradicts the SRS's own eligibility gate (§21.1); highest reputational/credibility risk in the project	Remove specific numbers; keep concept-only mock	Immediately
P0	No case/intervention workflow	This is the actual product differentiator; without it SAAP is a reporting layer, not a decision-support platform	Design the entity + minimal status flow	Design now, build Sem 5
P1	Missing `RolePermissions`, curriculum, and teaching-allocation tables	BR-003 and dynamic permissions are currently unenforceable at the data layer	Add 3–4 tables	Before Sem 4 migrations
P1	SCR-014 as a standalone destination	Presents a nonexistent model as if browsable/real	Replace with eligibility-checklist placeholder	Before next external review
P2	SCR-007/008 genericness	Low differentiation, not incorrect	Tie each to a role-specific decision	Sem 5
P2	SCR-016 premature telemetry	Monitors a system that doesn't exist yet	Shrink to minimal activity panel	Sem 4

Priority	Problem	Why	Action	Phase
P3	Persona picker on login screen	Fine for demo, wrong for anything else	Explicitly flag as prototype-only, remove in Sem 4	Sem 4
P3	No automated tests yet	Expected at this phase per SRS's own testing plan	Start in Sem 4 alongside backend	Sem 4

PART 21 — FINAL BRUTAL ASSESSMENT

1. **Is the current SAAP concept strong enough?** Not yet, as branded. As a role-scoped academic reporting tool, yes, it's a solid, competent Semester-3 deliverable. As a "decision-support / AI analytics platform," no — the decision-support half doesn't exist, and the AI half is currently make-believe dressed as a spec sheet.
2. **Is the current prototype convincing?** Visually and structurally, yes. Factually, no — two screens contain fabricated numbers indistinguishable from real ones, and that's a serious problem the moment anyone besides the team looks closely.
3. **Does it look generic?** The screen shapes (metric-card grid + chart + table) do, yes. The import-validation workflow and the mentor risk-flag logic do not — those are the two most defensible, least generic pieces of the whole build.
4. **Does it look AI-generated?** The prediction screen does, specifically — round, plausible, unfalsifiable numbers are exactly the tell.
5. **Is the current SRS too ambitious?** No — it is unusually well-calibrated about deferring ML and keeping humans in the loop. It is under-ambitious in one place: it never asks for a workflow/case-management layer, which is the piece that would make the "decision-support" framing true.
6. **Is the technical architecture appropriate?** Yes, for the phase and the scope. No red flags in the proposed FastAPI/PostgreSQL design.
7. **Is the database design appropriate?** Mostly — solid core, with 3–4 identifiable and fixable gaps, already half-diagnosed by the team's own review.
8. **Is the ML direction justified?** Not yet, by the SRS's own stated rule. The direction (a simple binary risk flag, human-reviewed) is reasonable; the current execution (a named, versioned classifier with invented metrics) is ahead of the data and should be pulled back, not pushed forward.

9. **Biggest weakness:** the gap between "detects a signal" and "does something about it" — nothing in the current design closes that loop.
10. **Biggest opportunity:** building the case/intervention layer is a moderate amount of additional work (one new entity, a status field, a handful of API endpoints) for a disproportionate increase in how real the "decision-support platform" claim becomes.
11. **What should absolutely NOT be done:** don't add more dashboards, more metric cards, or a "real-looking" ML model before real historical outcome data exists — every one of those moves makes the credibility gap (Part 13) worse, not better.
12. **What should be redesigned before Semester 4:** the two fabrication issues (P0s in Part 20), the ML model card, and the database's missing junction/mapping tables.
13. **What can remain untouched:** the import workflow, the mentor dashboard's core logic, the dependency-light frontend approach, the role/route mapping.
14. **What would be done differently starting today:** design the case/intervention entity and workflow *before* writing a single line of Semester 4 backend code, so the CRUD build in Semester 4 is built around the real product loop from day one instead of retrofitting it in Semester 5.

No numeric score is given — the SRS itself asks not to inflate one, and a single number would hide the actual finding, which is: **the engineering discipline is ahead of the product thinking, and the product thinking is ahead of the data honesty.** Fix the data honesty first — it's the cheapest fix and the highest-consequence one — then use the freed-up confidence to actually design the intervention loop that would make the "decision-support platform" claim true.

APPENDIX A — Current vs. Recommended Screen Map

Current (16 screens)	Recommendation
SCR-001 Login	Keep; flag persona picker as prototype-only
SCR-002 Student Dashboard	Keep, fix fabricated metrics
SCR-003 Academic Profile	Keep as-is
SCR-004 Marks View	Keep as-is
SCR-005 Attendance View	Keep as-is
SCR-006 Mentor Dashboard	Keep; upgrade notes → cases
SCR-007 Faculty Analytics	Merge toward a specific decision or fold into SCR-008 with a role filter
SCR-008 Department Analytics	Keep, fix fabricated pass-rate metric
SCR-009 Import Center	Keep as-is
SCR-010 Import Validation	Keep as-is
SCR-011 Master Data	Keep as-is
SCR-012 User Management	Keep as-is
SCR-013 Insights	Redesign as case-creation entry point
SCR-014 Prediction Analytics	Replace with eligibility-checklist placeholder until Sem 6
SCR-015 Reports	Keep as-is
SCR-016 Audit/System Health	Shrink to minimal activity panel until real backend exists
(new) Case/Intervention Detail	Add — the single highest-value net-new screen this review recommends

APPENDIX B — Current vs. Recommended Database

Current: 27 entities across Security/RBAC, Academic Master Data, Student/Mentorship, Assessments/Marks/Attendance/Results, Import, Analytics/ML, Reporting/Audit.

Recommended additions (all additive, none removed):

- `RolePermissions` (Roles ↔ Permissions junction)
- `CurriculumMap` (Program × Semester × Subject)
- `TeachingAllocations` (Faculty × Subject × Batch × Semester)
- `InterventionCases` (or minimally `MentorNotes`, if the fuller case-management redesign in Part 5/17 is not adopted this cycle)

APPENDIX C — Current vs. Recommended Tech Stack

No changes recommended to the SRS's proposed stack (React, FastAPI, PostgreSQL, Pandas/NumPy, scikit-learn when eligible, Docker). The only stack-adjacent recommendation is sequencing: don't introduce ML libraries into the codebase until the Semester 6 data-sufficiency review (Part 12) actually passes.

APPENDIX D — Requirements That Must Be Clarified (institutional decision needed)

1. Who owns a case/intervention once created — the mentor only, or can faculty/department also open one?
2. What happens to a case if a mentor assignment changes mid-semester?
3. Is a 75% attendance baseline meant to become department-configurable (per the team's own noted assumption), and if so, who is authorized to change it?
4. Should students ever see their own risk classification directly, or only the translated, evidence-linked insight (the team already leaned toward the latter — worth formally ratifying)?
5. What counts as "sufficient authorized historical data" for §21.1's ML gate — a specific minimum cohort count and semester span should be written down now, not decided informally when Semester 6 arrives.

APPENDIX E — Items That Must Not Be Built (yet)

- Any real ML training pipeline, before the data-sufficiency review in Semester 6 actually passes.
- Any automated academic decision (probation flags that trigger real consequences, auto-generated warnings sent to students) — human-in-the-loop only, permanently, per BR-009.
- A general-purpose BI/self-service query builder — out of scope for a student-success platform; a different product.
- Any additional "AI-sounding" screen or model card until the two existing fabrication issues (Part 14, 20) are fixed — adding more before fixing what exists would make the credibility problem worse, not better.

This review is based on the SRS document and the Semester-3 implementation ZIP as provided. No files were modified, no implementation code was written, and no Semester-4 work was started, per the review's own instructions.